SECTION A:-

CONCEPT APPLICATION

(1) Explain why Rest-based AI API are suitable for Java backend integration.

Rest-based AI APIs use standard HTTP methods (GET, POST, PUT, Delete) and exchange data in JSON format, making them easy to integrate with Java frameworks like Spring Boot.

Java provides libraries such as RestTemplate, WebClient, & HTTPClient to send & request & receive AI-generated responses efficiently

[2] Explain how JSON parsing fits on AI response handling:

AI API usually returns response in JSON format. Java application parse this JSON using libraries like Jackson or Gson to extract useful information. and convert into Java Objects for futher processing

[3] Explain how environment variables protect API Keys.

Environment variables store sensitive information, such as API Keys outside the application source code. This prevents Keys from beign exposed in version control system like Git, improves security, and allows different keys to be used.

[4] Explain how spring Boot Rest endpoints expose AI response

Spring Boot Rest controllers receive client requests, forward them to the API through a service layer, parse the returned response and expose the AI-generated output as JSON through REST endpoints. This allows frontend applications to consume AI service using simple HTTP requests.

[5] Explain how prompt template improves chatbot's consistency.

Prompt templates provide a predefined structure for user requests by including consistent instructions, context and formatting. This helps the AI generate more accurate, predictable, and uniform responses while reducing ambiguity and improving the overall user experience

[6] Explain error handling strategies when AI API fails.

Catching exception with try-catch blocks.
Returning meaningful error message to users.
implementing retry mechanisms for temporary failures.
setting request timeouts to prevent long waits.
Logging errors for debugging and monitoring.